

Optimization of a UAV for Wildfire Management

Nathaniel S. Hargan

Dissertation submitted to the Faculty of the
Virginia Polytechnic Institute and State University
in partial fulfillment of the requirements for the degree of

Master of Science
in
Mechanical Engineering

Robert L. West Jr., Ph.D., Chair

Michael Philen

Pinar Acar, Ph.D

December XYZ, 2025

Blacksburg, Virginia

Keywords: Some Keywords, Subject matter, etc.

Copyright 2025, Nathaniel S. Hargan

Optimization of a UAV for Wildfire Management

Nathaniel S. Hargan

(ABSTRACT)

Give a brief description of your thesis here.

Optimization of a UAV for Wildfire Management

Nathaniel S. Hargan

(GENERAL AUDIENCE ABSTRACT)

You are also required as of Spring 2016 to include a general audience abstract. This should be geared towards individuals outside of your field that may be reading seeking information about your work. You should avoid language that is particular to your field and clearly define any terms that may have special meaning in your discipline.

Contents

List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 One Section	1
1.1.1 A sub-section	1
1.2 Another Section	1
2 Review of Literature	2
3 UAV Design	3
3.1 UAV Geometry	3
3.1.1 Design Variables	4
3.1.2 Design Parameters	4
3.1.3 Material Properties	6
3.2 Propellers and motors	7
3.3 Power	7
3.4 Mission Profile	7

3.4.1	Thrust Calculation	9
3.5	UAV Power	10
3.5.1	Power Calculation	10
3.5.2	Energy Consumption Calculation	10
3.5.3	Percent Throttle Calculation	11
3.5.4	Perturbation Calculation	11
4	Finite Element Analysis	17
4.1	Beam Element Formulation	18
4.1.1	Beam Element Stress	19
4.2	UAV Segment Model	19
4.3	Solvers	20
4.3.1	Static	20
4.3.2	Dynamic	20
4.3.3	Transient	20
4.4	Optimization	21
4.4.1	Objective Function and Design Variables	24
4.4.2	Optimization parameters	24
4.4.3	Constraints	24
5	Results	31

6	Conclusions	32
6.1	Further Work	32
7	Summary	33
	Bibliography	34
	Appendices	35
	Appendix A First Appendix	36
A.1	Section one	36
A.2	Section two	36
	Appendix B Second Appendix	37

List of Figures

3.1	Optimized UAV Model with Highlighted Arm	3
3.2	Optimized UAV Model with Highlighted Strut	4
3.3	Optimized UAV Model with Highlighted Strut	4
3.4	Optimized UAV Model Hub Cross-section View	5
3.5	Design Variable Visualizations on UAV model	13
3.6	Downrange Position vs. Altitude	14
3.7	Altitude vs. Time	14
3.8	Downrange position vs. Time	15
3.9	Thrust vs. Time Plot	15
3.10	Power Vs. Time Chart for Optimized UAV	15
3.11	Throttle Vs. Time Chart for Optimized UAV	16
3.12	Perturbation Vs. Time Chart for Optimized UAV	16

List of Tables

3.1	Material Properties for Braided Round Tubes from ALLRed Corporation [5]	6
3.2	Mission Profile Constraints	8
5.1		31

NLP is a field of computer science, artificial intelligence, and linguistics concerned with the interactions between computers and human (natural) languages.

σ is the eighteenth letter of the Greek alphabet, and carries the 's' sound. In the system of Greek numerals, it has a value of 200.

Chapter 1

Introduction

1.1 One Section

1.1.1 A sub-section

1.2 Another Section

Chapter 2

Review of Literature

Chapter 3

UAV Design

The UAV design being optimized is based from the UAV design in Nick James Newton's Master's Thesis "Development of Tools for Conceptual Design of a Wildland Firefighting UAV"

3.1 UAV Geometry

The UAV is modeled as a simplified octocopter frame. The UAV geometry is shown in figure 3.1 - figure 3.3 with the UAV frame components highlighted. The frame model contains 3 components: the arms, the struts, and the hubs. The arms and struts are both beams with annular cross sections. The hub is two octagonal plates connected with a fanning, radially symmetric web shown in figure 3.4b.

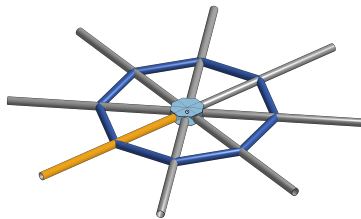


Figure 3.1: Optimized UAV Model with Highlighted Arm

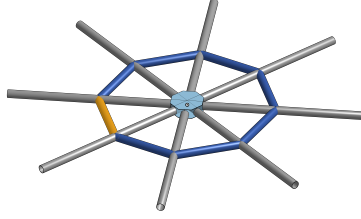


Figure 3.2: Optimized UAV Model with Highlighted Strut

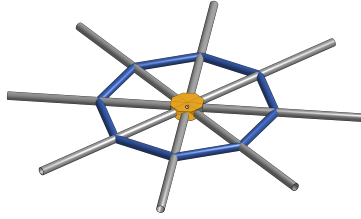


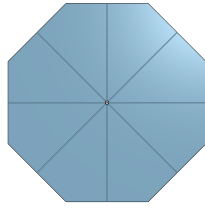
Figure 3.3: Optimized UAV Model with Highlighted Strut

3.1.1 Design Variables

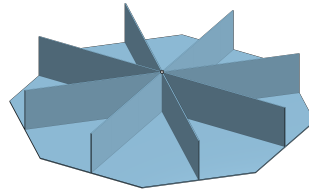
There are 8 design variables that were selected for this analysis. Each design variable is shown in Figure 3.5. The two design variables associated with the 8 arms of the UAV frame are the arm outer diameter (d_a) and arm wall thickness (t_a). The design variables associated with the eight struts are the strut outer diameter (d_s), strut wall thickness (t_s), and the distance from the center of the UAV to the ends of the struts (r_s). The design variables associated with the hub are the flange thickness (r_h), the hub web thickness (t_w), and the hub radius (r_h).

3.1.2 Design Parameters

There are two other parameters related to the UAV frame geometry dimensions that were not selected as variables. The wingspan and the hub height.



(a) Top Section View



(b) Isometric View

Figure 3.4: Optimized UAV Model Hub Cross-section View

The inner hub height is a parameter set to 64mm, which is the near the minimum height required to fit the UAV battery between the top plates. The actual batteries would not fit on this frame model, because the hub web would intersect the battery. Further work could be done refining the mesh to allow for more realistic battery storage.

The distance from the center of the UAV to the propeller is calculated to be 1127.3mm. This is calculated by finding the minimum distance required to fit all 8 propellers with a radius of 406.4mm with a propeller spacing of 25mm.

The UAV carries a payload of 45kg.

The DroneGeometry class within the UAV_Geometry module stores relevant design variables for a drone geometry. The relevant coordinates for hub, arm, and struts are calculated. The projected surface area for drag calculation a drone geometry is calculated using the `calc_projected_surface_area` method. [talk about this in detail later]

3.1.3 Material Properties

The material selected for all UAV frame components (struts, arms, and hub) is braided carbon fiber composites. Due to the lack of available data, carbon fiber is treated as isotropic. Further work could be done to acquire more accurate data for the material. The carbon fiber data in Table 3.1 is for braided round tubes sourced from ALLRed corporation.

Table 3.1: Material Properties for Braided Round Tubes from ALLRed Corporation [5]

Young's Modulus	Gpa	19
Ult. Tensile Strength	Mpa	345
Poisson's Ratio		0.3
Mass Density	lb/in^3	0.044

The shear strength is calculated from the relationship tensile strength and poisons ratio between the shear strength.

Due to lack of fatigue information for the specific carbon fiber composites, material data is taken from Aluminum7075-T6. [3] The fatigue strength coefficient (f') for Aluminum7075-T6 is 1466 MPa. The fatigue strength exponent (b) is -0.143. The fatigue strength cycles is $1e6$ (N). The completely reversed stress amplitude was calculated from the equation $\sigma_{rev} = f'(2N)^b$

All data for materials is stored within the Material dictionary within the material_properties module.

A Woehler curve for the fatigue analysis as created using the Pylife Library. [4] The completely reversed stress amplitude is input as 'SD', the fatigue strength cycles for 'ND', and the fatigue strength esponent for 'k_1'. This was used for the WoehlerCurve class.

3.2 Propellers and motors

3.3 Power

The battery used is the Tattu HV 40000mAh 10C 6S1P 22.8V High Voltage UAV Lipo Battery Pack [2]. Two of these batteries are in series. The batteries have 40000mAh and supply 22.8V. The total voltage of the power system is 45.6V. The energy capacity ($E_{Capacity}$) of the battery system is calculated by multiplying the voltage of the entire system by the amp-hours of the battery. Battery power data and related calculation are stored within the BatterySystem class within the drone_power module.

The batteries have the dimensions of 263x122x64mm. The centroid of the batteries are positioned at 0, 86.75, 0 and 0, -86.75, 0 for the mass analysis. The mass and position data for the batteries are stored within the Battery class within the drone_battery module.

3.4 Mission Profile

A model of the mission profile was created to describe a potential mission with continuous position, velocity, and acceleration with respect to time. The mission profile is modeled as a series of mission profile segments. Constraints are put on the position, velocity, and acceleration at the nodes between mission profile segments. The remaining values at the nodes are calculated using basic kinematic equations. At values of time between mission segment nodes, the position, velocity, and acceleration are interpolated using cubic Hermite splines. [citation needed] Acceleration changes linearly with respect to time.

The mission profile is based on a potential mission during the 2025 Palisades wildfire. The distance for the mission profile is estimated to be 5600m, the distance from the north edge

of the active fire area to south edge of the wildfire. Using elevation maps of the surrounding area, the height required to transverse this mission path is 460m. [cite map] Timeesteps were set so that maximum x-velocity to be under 12m/s. [cite nick thesis?]

The climb, cruise, and descent periods are each described using two separate cubic Hermite spline intervals to prevent over-defining the mission profile. Figure 3.6 shows the altitude for the mission profile against the down range position. Figure 3.7 shows the altitude over time. Figure 3.8 shows the downrange position with respect to time. Table 3.2 shows the constraints on the mission profile. The duration shows the time between spline intervals. Blank entries are where no constraint was applied.

Table 3.2: Mission Profile Constraints

Segment	Initial	Takeoff	Climb 1	Climb 2	Cruise 1	Cruise 2	Decent 1	Decent 2	Land
Duration (s)	N/A	12	10	10	25	25	10	10	15
Position X (m)	0			500	2800	5100			5600
Position Y (m)	0			560	560	560			0
Velocity X (m/s)	0								0
Velocity Y (m/s)	0								0
Acceleration X (m/sec^2)	0			0		0			0
Acceleration Y (m/s^2)	0			0		0			0

Mission profiles are generated with a mission_profile module. This module contains a MissionProfile class, MissionProfileSegment class, and the kinematic matrix function. The MissionProfile contains a list of constraints added by the add_constraint method, and a list of segment durations added by the add_segment method. It also contains the solve_kinematics and time_solve method. The solve_kinematics method assembles matrixes generated from the kinematic_matrix function into a kinematic matrix for the entire mission profile. The remaining position, velocity, and acceleration values at the nodes are then calculated using the numpy.linalg.lstsq function. The time_solve method uses a cubic Hermite spline to find interpolate values at a given time.

3.4.1 Thrust Calculation

A calculation of the thrust profile is required in order to perform the power and fatigue analysis.

The thrust equation over the duration of the mission profile is shown below. Both the acceleration and drag forces are considered.

$$F_{\text{thrust}}(t) = + - v(t)^2 * s_a * \rho * c_d + (a(t) + g) * m$$

The $v(t)$ and $a(t)$ represent the velocity and acceleration of the mission profile at given point in time.

s_a is an estimate of the surface area of UAV. This is obtained by summing the projected surface of every beam and hub segment with the projected surface area of the two batteries. It is a function of the design variables.

$\rho = 1.2250 \text{ kg m}^{-3}$ is the air density. $c_d = 0.256$ is the drag coefficient. [How to justify?] $g = 9.8066 \text{ m s}^{-2}$ was used for the gravity constant.

The mass is estimated as the sum of 45kg payload, the mass of the frame, and the mass of the two batteries.

To simulate the pre-liftoff, and post-landing period, a rev up and rev down period were added to the thrust forcing function. This was modeled as a linear increasing force with respect to time up until the force required for take off for the rev up period. The rev down period was modeled as a linearly decreasing force until no force is applied. A duration of 5 seconds was selected for both periods.

3.5 UAV Power

The batteries in the design are two "Tattu 40000mAh 6S 10C 22.8V High Voltage UAV Lipo Battery Pack with AS150+AS150 Plug" [I don't know how to format this] They have 40000mah and supply 22.8 volts of power.

Power calculations are done with the DronePower class within the drone_power module. The class takes battery data, the thrust profile, and a timestep as inputs to calculate the consumed energy and power consumption over the duration of the mission.

3.5.1 Power Calculation

A model of the drone power system was developed to describe the power demand of the vehicle throughout the mission. The trust at a given time is converted to power based on experimental data. [CITE] This is interpolated as a quadratic function. This allows the power draw to be calculated at any point during the mission.

Equation 3.5.1 shows the resulting thrust to power requirement.

$$P(t) = 0.01238T(t)^2 + 0.03214T(t) + 0.06548$$

3.5.2 Energy Consumption Calculation

The energy consumed over the entire mission duration was taken by subtracting the integral of the power draw with respect to time from the batteries overall energy capacity. Equation 3.5.2 describes the relationship between the energy remaining in the battery and the power

consumed. $E_{Remaining}(t)$ is the total energy remaining between both batteries as a function of time. $E_{Capacity}$ is the energy stored by a fully charged battery. t is the time since the start of the mission.

$$E_{Remaining}(t) = E_{Capacity} - \int_0^t P(t)$$

The integral of the power is calculated using `scipy.integrate.cumulative_trapezoid` to find the energy consumed. The number of timesteps was set to a finite number and is a variable set when running the optimization function.

3.5.3 Percent Throttle Calculation

The percent throttle function is calculated as a power required divided by the power required for maximum throttle. P_{max} is the power required for max throttle. [According to what? Cite paper? Justification pleas?]

$$T_{percent}(t) = P(t)/P_{max}$$

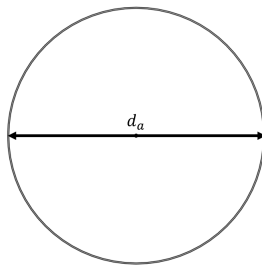
3.5.4 Perturbation Calculation

For the transient damage analysis a model of oscillations caused by propeller perturbations was also required. This as modeled as a force over time. The desired function is a sinusoidal wave with a frequency matching the propeller frequency. The amplitude was estimated to be

1% of the thrust. Equation 3.5.4 shows the completed equation for the perturbation forcing function. ω_{max} is the maximum angular velocity of the propeller.

$$F_{\text{perturbation}}(t) = 0.01 \sin(\omega_{max} T_{percent}(t) t) F_{\text{thrust}}(t)$$

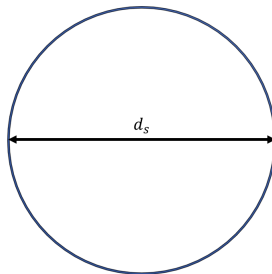
Figure 3.12 shows the perturbation force over time for the optimized UAV model.



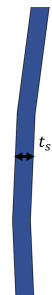
(a) Arm Diameter



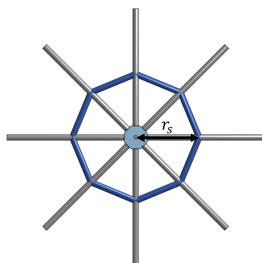
(b) Arm Thickness



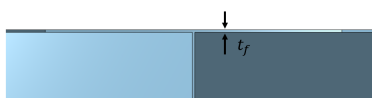
(c) Strut Diameter



(d) Strut Thickness



(e) Strut Distance from center



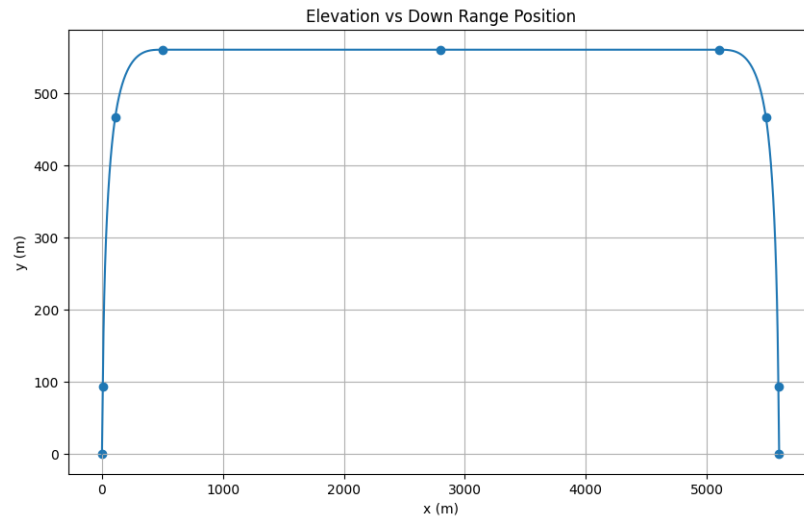


Figure 3.6: Downrange Position vs. Altitude

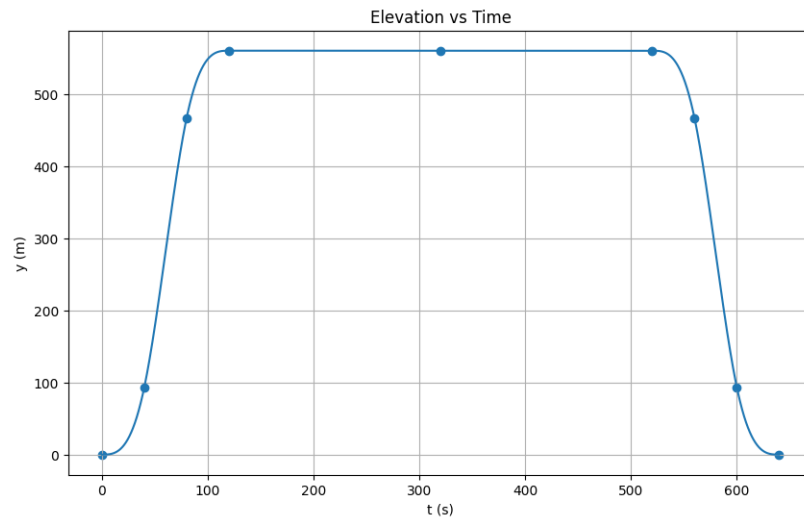


Figure 3.7: Altitude vs. Time

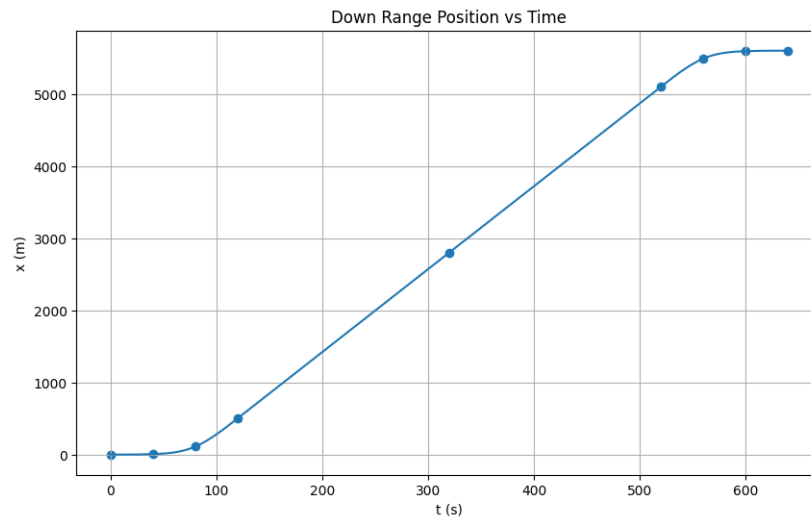


Figure 3.8: Downrange position vs. Time

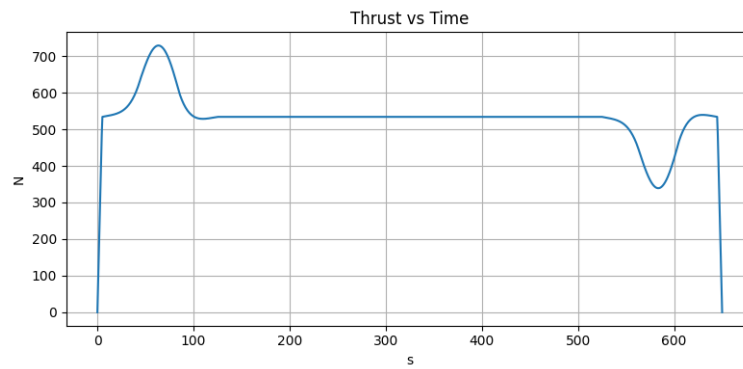


Figure 3.9: Thrust vs. Time Plot

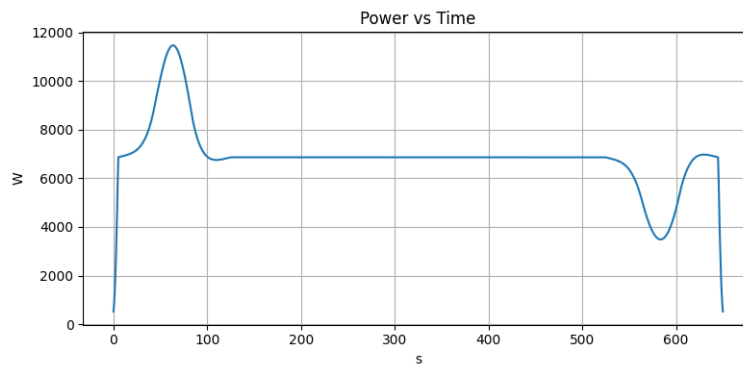


Figure 3.10: Power Vs. Time Chart for Optimized UAV

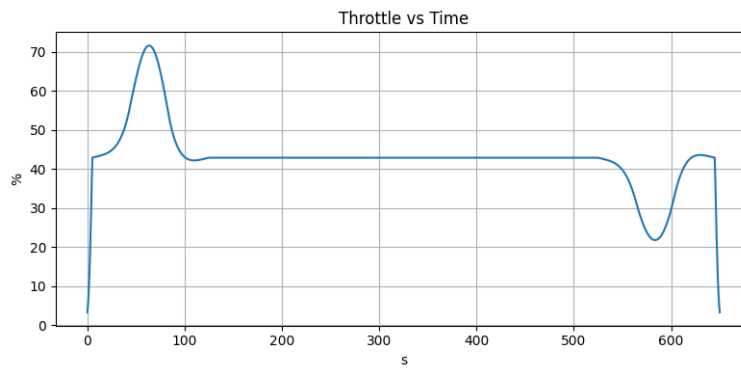


Figure 3.11: Throttle Vs. Time Chart for Optimized UAV

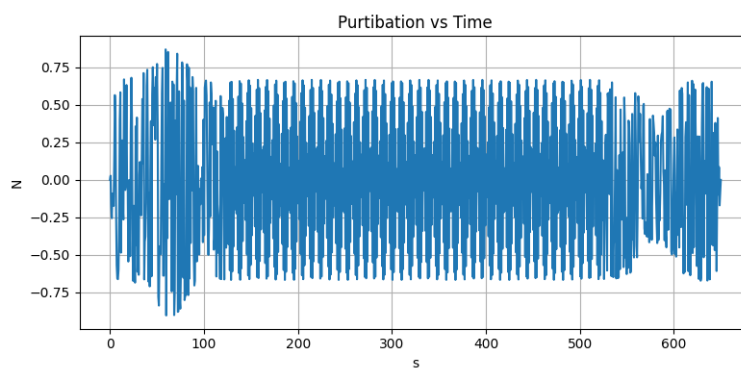


Figure 3.12: Perturbation Vs. Time Chart for Optimized UAV

Chapter 4

Finite Element Analysis

A finite element model was created to be run repeatedly by the optimization algorithm. The finite element model is designed so that it can be easily adapted for different frame models

- Describe the matrix assembly process? (mass and stiffness)

The finite element model is divided into several different modules. The `beam_system` module contains the `BeamSystem` class. The `beam_type` module contains `BeamType` class which stores beam cross section and material definitions. The `beam_element` module contains information about each beam element in the `BeamElement` class. The `node` module contains the `Node` class.

A material property dictionary is stored in `material_properties` module. Cross section properties are calculated using functions in the `cross_section_properties` module.

The `BeamSystem` class contains list of `BeamElement` objects and a list of `Node` objects. The `BeamElement` object. Each `BeamElement` object references a `BeamType` object. The `BeamType` object stores the material properties retrieved from the `material_module`. The cross section properties are calculated for a given cross section using functions within the `cross_section_properties` module.

Nodes are added using the `add_node` method in the `BeamSystem` class. The nodes contain a single position in the global coordinate system.

Beam elements are added to the BeamSystem class using the add_beam method. The add_beam method requires a reference to a start and an end node, a BeamType object and a k node to determine the orientation of the beam's cross-section.

The boundary conditions are added with the add_boundary_condition method. The method requires the value of the boundary condition, the boundary condition type (x, y z, x rotation, ect.), and node reference here the boundary condition is applied.

The solve_FEA method solves the stiffness and mass matrixes for the given system. The solve_static, solve_dynamic_harmonic, and solve_transient methods find the solution for given forces.

4.1 Beam Element Formulation

Beams are modeled as timeshenko beams elements with 12 degrees of freedom [citation needed?]. This beam model takes shear deformation effects into account. The mass for each beam is modeled as a 3D lumped mass matrix.

- More detail on mass matrix (it includes rotations)

Boundary conditions are set relative to a local coordinate system so that the symmetric boundary conditions at the endpoints of the struts.

Stresses and strains are calculated using the formula ??? [cite presentation]

Each beam element within the beam system is associated with a BeamElement object. The local stiffness and mass matrices are calculated with the solve_stiffness_matrix method.

Intrmediate values for displacements are found using the return_shape_functions, return_shape_functions and return_shape_functions_dynamic method. Intermediate values of stress and strain are

found using the `stress_strain_shape_function` method.

4.1.1 Beam Element Stress

[Discuss the stress shape function method?] [Cite the lecture idk]

Beam element failure is calculated for each element. The stress shape function for the beam is calculated and the maximum stress state between the starting node and the ending node of each element is found. Then the maximum between the two nodes is the maximum stress.

Stresses and strains are calculated with the `find_stresses`, `find_stresses_dynamic` and `find_stresses_transient` methods. The failure criterion is calculated in the `calc_failure_criterion_swt` method within the `BeamElement` class.

4.2 UAV Segment Model

A FEA model of single slice of the UAV frame was created. There are symmetric constraints at the midpoints of the strut beams. The center of the UAV is fixed in all 6 degrees of freedom. This assumes that the modes of frequency are symmetric. [maybe write about an asymmetric frequency model if I do that analysis] The arm and struts have an annular cross section. The hub is represented as a series of repeated I-beam cross sections. The width of each I-beam cross-section in the FEA model is the width at the centroid of the corresponding trapezoidal area of the hub.

The UAV segment model is generated in the `DroneFEA` class within the `drone_fea` module. This module requires a `DroneGeometry` object as input. A `BeamSystem` object is generated to perform the FEA analysis. The nodes are generated using the `create_drone_slice_node` method which adds nodes to the `BeamSystem` object. The `create_drone_slice_beams`

method calculates the hub section parameters and adds the beams to the BeamSystem object. The boundary_conditions_slice method applies the boundary conditions to the beam model.

4.3 Solvers

An FEA model was created with three separate solvers. The static solver, which is used to calculate the response given a static loading, the dynamic solver, which calculate the response given a force of an oscillating force at a given frequency, and the transient solver which calculates the response given a time-changing force.

4.3.1 Static

The static solver uses point loads calculate - Outputs - Stress - Explain how stress calculation is obtained - calculated from the local displacement in each element - Strain - Explain how strain calculation is obtained - calculated from the local displacement in each element - Natural Frequency

4.3.2 Dynamic

- Inputs - Point Loads - Frequency [Cite source] - Outputs - Stress Amplitude - calculated from the local displacement in each element - Strain Amplitude - calculated from the local displacement in each element - Natural Frequency

4.3.3 Transient

- Inputs Function describing a force at a given time - Uses Newmark-Beta integration to calculate resulting transient displacements and reactions - The transient stress is obtained from the transient displacements and reactions

4.4 Optimization

design variables:

- d_a : arm diameter,
- t_a : arm thickness,
- d_s : strut diameter,
- t_s : strut thickness,
- r_s : strut distance from center,
- t_f : hub flange thickness,
- t_w : hub web thickness
- h_r : hub radius

derived variables:

- d_{tip} : Deflection at tip
- σ_{max} : Maximum stress from bending

- f_{amp} : Amplitude of displacement at the tip of the uav arm
- D : Damage per mission
- E_{used} : Energy used over th duration of the mission
- E_{total} : Energy used over th duration of the mission

design parameters:

- $d_{allowable}$: Deflection allowable
- $\sigma_{allowable}$: Stress allowable from bending
- $R_{allowable}$: Allowable amplitude displacement ratio
- $n_{missions}$: number of missions before damage

- Optimization algorithm used is SLSQP method using scipi's minimization function.

4.4.1 Objective Functions

The problem is formulated as follows:

$$\min \text{mass}(d_a, t_a, d_s, t_s, r_s, t_f, t_w)$$

subject to constraints:

$$g_j(x) \leq 0; j = 0, 1, 2, \dots, J$$

$$x_i^L \leq x_i \leq x_i^U$$

The objective function is to minimize the mass of the vehicle. The mass is calculated in the DroneMass class within the drone__mass module. The DroneMass class takes a DroneGeometry object as an input. It uses the material information, cross-section properties, and Beam and hub coordinates within the DroneGeometry object to calculate the mass of the UAV frame.

4.4.2 Constraints

Boundaries

Each design variable has a boundary condition applied to it. Arm diameter Equations 4.4.3 - 4.4.3 describe each boundary condition. The arm diameter (d_a), arm thickness (t_a), strut diameter (d_s), strut thickness (t_s) boundary conditions were obtained through the minimum and maximum pipe diameter and thickness for braided carbon fiber found on AllRED's Dragonplate Website. [1]. The hub flange thickness (t_f) and hub web thickness (t_w) boundary conditions were taken from the minimum and maximum thickness of carbon fiber sheets. The maximum boundary conditions on hub radius (h_r) and strut distance from the center (r_s) are the distance from the center of the UAV to the propellers so the struts and hub stay within the frame. The minimum constraint of 197.5mm for hub radius (h_r) and strut distance (r_s) is the minimum hub radius for a circle to circumscribe both batteries with their given positions and dimensions.

- d_a : arm diameter,
- t_a : arm thickness,
- d_s : strut diameter,
- t_s : strut thickness,

- r_s : strut distance from center,
- t_f : hub flange thickness,
- t_w : hub web thickness
- h_r : hub radius

$$9\text{mm} \leq d_a \leq 50.0\text{mm}$$

$$0.254\text{mm} \leq t_a \leq 3.175\text{mm}$$

$$9\text{mm} \leq d_s \leq 50.0\text{mm}$$

$$0.254\text{mm} \leq t_s \leq 3.175\text{mm}$$

$$197.5\text{mm} \leq r_s \leq 1127.3\text{mm}$$

$$0.7\text{mm} \leq t_f \leq 12.7\text{mm}$$

$$0.7\text{mm} \leq t_w \leq 12.7\text{mm}$$

$$197.5\text{mm} \leq h_r \leq 1127.3\text{mm}$$

Displacement

The static FEA solver was used to calculate the maximum displacement magnitude. To simulate the loading at hover, the loading is located at the end of the UAV arm pointing upward while the center of the UAV is fixed in displacement and rotation. The load is a point load and is equivalent to the mass of the payload multiplied by gravity divided by the number of blades.

The formula for the constraint is shown in 4.4.3. d_{max} is the maximum displacement of the UAV frame with the described loading. $d_{allowable}$ is the maximum allowable displacement,

$$g_1(d_a, t_a, d_s, t_s, r_s, t_f, t_w) = d_{max} \leq d_{allowable}$$

Stress

The loading described for the displacement constraint was also used for the stress constraint. The static FEA solver was used. The allowable stress is determined by the maximum stress of the material and a factor of safety.

σ_{max} is the maximum nodal bending stress in the FEA beam system as calculated by the . σ_{yield} is the tension yield strength of the material. FOS is the Factor of Safety associated with the stress constraint.

$$g_2(d_a, t_a, d_s, t_s, r_s, t_f, t_w) = \sigma_{max} \leq \sigma_{yield} * \text{FOS}$$

Frequency

The objective of the frequency constraint is to ensure that the frequency experienced by the propeller is not near the natural frequency of the vehicle.

Two analyses were done. The static analysis and the frequency analysis. The static analysis is equivalent to the on displacement constraint. A frequency analysis was done with a amplitude equivalent to the magnitude of the force in the static analysis. Formula 4.4.3 describes the ratio of the amplitude of displacement ($d_{amplitude}$) in the dynamic analysis to the resulting displacement of static analysis (d_{static}). The should be less than the allowable ratio ($R_{allowable}$). The magnitude of these force could be any number, the payload was chosen so the static analysis could be reused, saving computation time. [Justify??? Source???]

$$g_3(d_a, t_a, d_s, t_s, r_s, t_f, t_w) = \frac{d_{amplitude}}{d_{static}} \leq R_{allowable}$$

The frequency of the oscillating loading function is described as two times the maximum rad/s of the propeller times the maximum percent throttle experienced during the mission profile. The forcing function is assumed to be twice propeller speed due to the propeller having two blades.

This is used for the frequency of the oscillating force for the dynamic solver. The location of the force and boundary conditions are the same as the static analysis.

Damage

The damage constraint is imposed in order to limit damage caused by the vibrations in the vehicle. The finite element transient solver is used for this constraint. The force is applied to

the propeller of the sector model. The boundary conditions are the same in this constraint. The thrust required to achieve the mission profile ($F_{\text{thrust}}(t)$) discussed in subsection 3.4.1 is added to the perturbation function ($F_{\text{perturbation}}(t)$) described in subsection 3.5.4. This function is divided in eight because it is applied to the sector model.

The mission duration is divided into equal time-steps according to the $N_{\text{transient_timesteps}}$ parameter. The number of time-steps for the transient solver is $N_{\text{transient_timesteps}}$. The constants used for Newmark-beta integration are $\beta = 0.25$ and $\gamma = 0.5$. *The results for the maximum bending stress over the point Rain flow-counting algorithm to transfer the time-varying load into set of constant amplitude stress*

One limitation of this analysis is that material data is fatigue material data is limited to aluminum as fatigue data for the carbon fiber composite has not been obtained as that is outside the scope of this thesis.

$$g_4(d_a, t_a, d_s, t_s, r_s, t_f, t_w) = D * n_{\text{missions}} \leq 1$$

Energy

The objective of this constraint is to keep battery energy above desired levels for the duration of the mission. The energy constraint uses the energy consumption calculation in Subsection 3.5.2 to determine the total energy consumed during the mission. E_{used} is equal to $E_{\text{Capacity}} - E_{\text{Remaining}}(t)$ where t is the full duration of the power mission profile. E_{Capacity} is the total storage of the battery. $E_{\% \text{remaining}}$ is a parameter which is equal to the amount of percent energy remaining desired after landing.

$$g_5(d_a, t_a, d_s, t_s, r_s, t_f, t_w) = E_{used}/E_{Capacity} \leq E_{\%remaining}$$

4.4.3 Optimization parameters

Chapter 5

Results

- The initial guess were based off of measurements from the original UAV design. - arm diameter, strut diameter, hub radius, flange thickness, and web thickness were all found to converge to boundary conditions

-Sensitivity analysis - The value of the design variables which did not converge to the boundary conditions were varied by $\pm 0.1\%$ of their values to test the optimal solution.

Table 5.1

Chapter 6

Conclusions

6.1 Further Work

Take into account UAV tilt

Objective function with moment of inertia.

The UAV mesh can be refined

The hub model is a simplification, as it cannot fit the entire battery within the bounds of the hub. Further work can be done to create a more detailed model.

Asymmetric modes should be considered.

Chapter 7

Summary

Bibliography

- [1] URL <https://dragonplate.com/>.
- [2] URL https://genstattu.com/tattu-40000mah-6s-10c-22-8v-high-voltage-uav-lipo-battery-?srsltid=AfmB0oo89wjwe0S5x2-_3PrUxTjnsa3qDuCLy9eYPu2Dz_ErwfbAG77_.
- [3] URL <https://matweb.com/search/datasheet.aspx?matguid=4f19a42be94546b686bbf43f79c51b7d&ckck=1>.
- [4] URL <https://pylife.readthedocs.io/en/latest/README.html>.
- [5] BJ Taylor. Dragonplate enquiry.

Appendices

Appendix A

First Appendix

A.1 Section one

A.2 Section two

Appendix B

Second Appendix